Word Recognition System

Speech-Command Classification with MFCC + GMM

Advanced DSP — Project Report

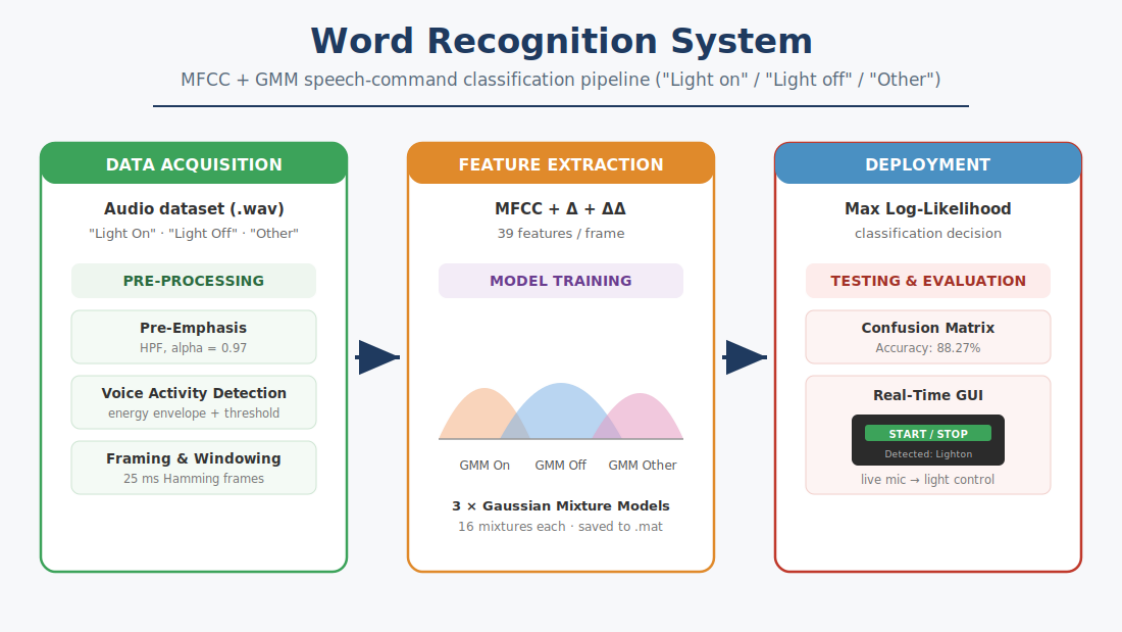


Figure 1. End-to-end system architecture

1. Overview

This project presents a word-recognition system that classifies short audio inputs into three categories: "Light On", "Light Off", and "Other". The recognised command drives a simulated light through a real-time graphical interface, demonstrating a complete voice-controlled pipeline from microphone to action.

The signal-processing pipeline consists of three main stages:

- **Pre-Processing —** Pre-Emphasis to boost high frequencies, Voice Activity Detection (VAD) to isolate speech, and Hamming windowing to divide the signal into short frames.
- **Feature Extraction —** Mel-Frequency Cepstral Coefficients (MFCCs) together with their first and second derivatives, giving 39 features per frame.
- **Model Training —** Gaussian Mixture Models (GMMs) that classify new audio based on maximum log-likelihood.

Dataset

The system was trained on 308 "Light On", 306 "Light Off" and 602 "Other" samples. Testing used a balanced set of 60 samples per category.

2. Signal Processing Pipeline

2.1 Configuration & Loading

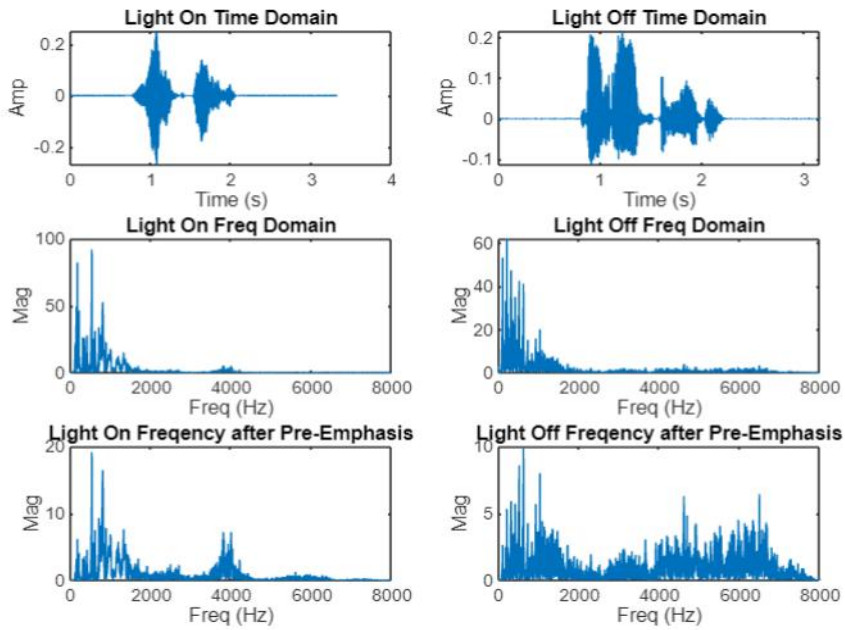
Audio files are read per category with audioread, preserving each file's native sample rate for downstream processing.

```
clc; clear; close all;

% Define paths
basePath = fullfile('Data', 'Train');
pathOn   = fullfile(basePath, 'LightOn');
pathOff  = fullfile(basePath, 'LightOff');

% Get first wav file from each folder
filesOn  = dir(fullfile(pathOn,  '*.wav'));
filesOff = dir(fullfile(pathOff, '*.wav'));

% Read audio files
[audioOn,  fsOn]  = audioread(fullfile(pathOn,  filesOn(1).name));
[audioOff, fsOff] = audioread(fullfile(pathOff, filesOff(1).name));
```

2.2 Pre-Emphasis

Pre-emphasis applies a first-order high-pass filter (alpha = 0.97) to the raw signal. This amplifies high frequencies that are naturally attenuated in human speech, producing a more balanced spectrum and improving the quality of the extracted features.

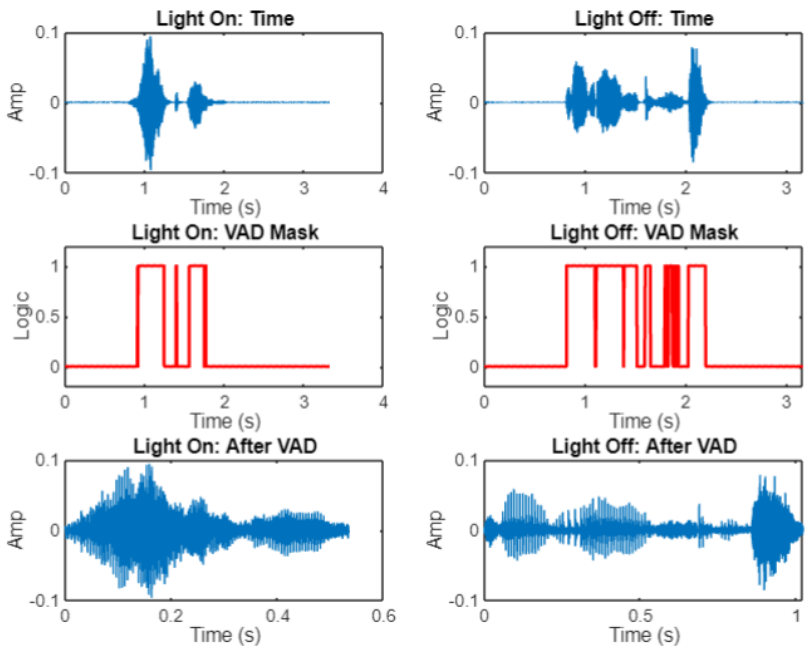
```
% Apply pre-emphasis
alpha = 0.97;
preEmphOn  = filter([1, -alpha], 1, audioOn);
preEmphOff = filter([1, -alpha], 1, audioOff);

% Compute pre-emphasized FFT
fftOn_pre  = abs(fft(preEmphOn));
fftOff_pre = abs(fft(preEmphOff));
```



2.3 Voice Activity Detection (VAD)

VAD isolates speech segments by removing silence and ambient noise. A moving average of the signal magnitude forms an energy envelope; frames whose envelope exceeds 10% of the peak are kept as active speech. This ensures the feature-extraction stage processes only relevant acoustic data and prevents the model from learning noise characteristics.

```
% Define window length for energy envelope
winLen = round(0.02 * fs);

% Calculate signal envelope using moving average
env = movmean(abs(preEmphSignal), winLen);

% Define threshold and build the VAD mask
thresh  = 0.1 * max(env);
vadMask = env > thresh;

% Keep only active speech
vadSignal = preEmphSignal(vadMask);
```

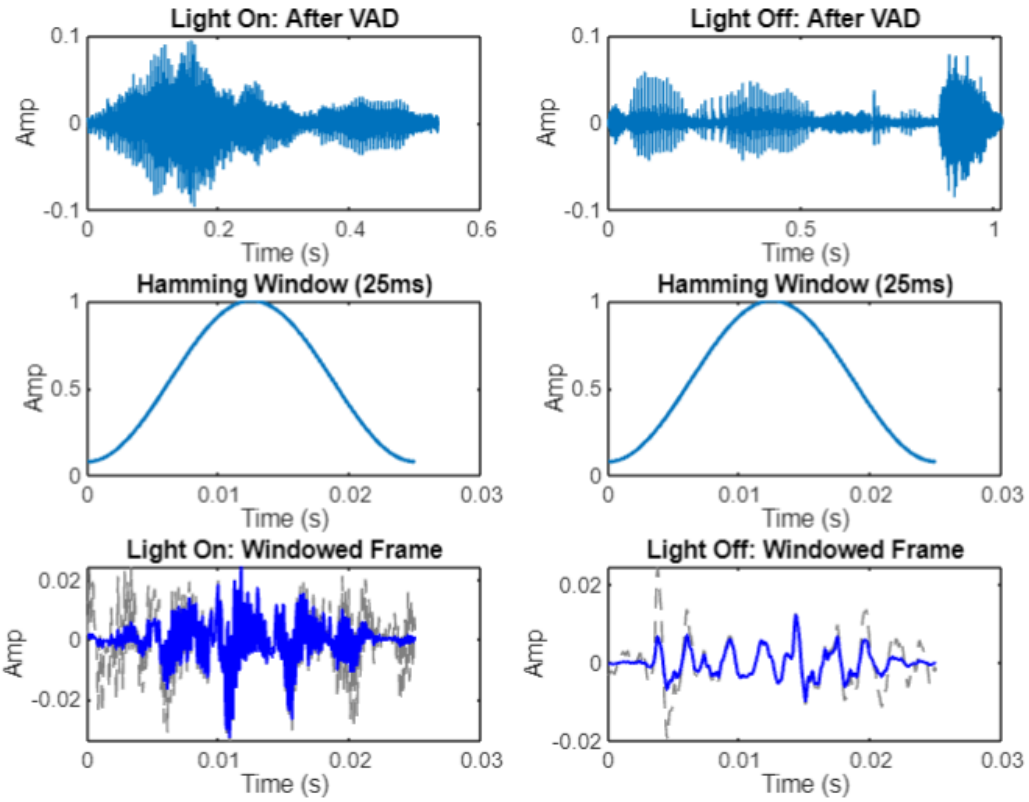


2.4 Framing & Windowing

Because speech is non-stationary, it is analysed in short (~25 ms) overlapping frames. Abrupt frame boundaries introduce spectral leakage, so a Hamming window tapers each frame toward its edges, yielding a more reliable frequency representation before the FFT.

```
% Define frame parameters
frameDuration = 0.025;              % 25 ms frames
```

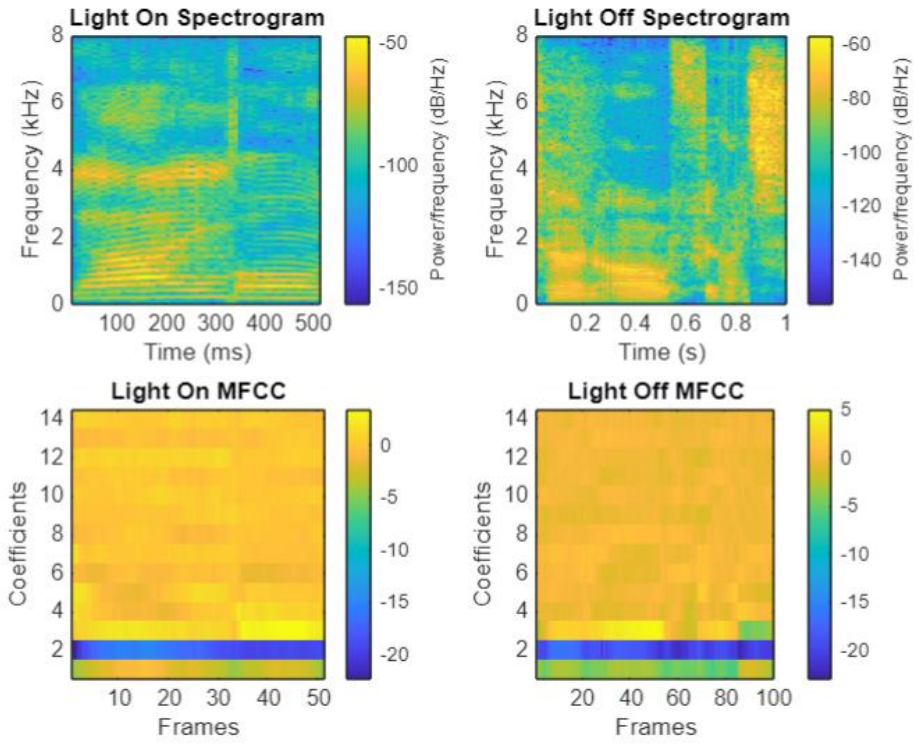
```
winLen = round(frameDuration * fs);

% Create Hamming window and apply it to a frame
hamWin   = hamming(winLen);
frameWin = frameOrig .* hamWin;
```



3. Feature Extraction (MFCC)

To represent voice commands mathematically, the system extracts Mel-Frequency Cepstral Coefficients. Each windowed frame is transformed to the frequency domain, passed through a Mel-scale filterbank, log-scaled, and reduced with a DCT. The first 13 coefficients describe the vocal-tract shape; appending Delta and Delta-Delta coefficients expands the representation to 39 features per frame, capturing how the spectrum evolves over time.

```
% Frame parameters
winLenMfcc  = round(0.025 * fs);     % 25 ms window
overlapMfcc = round(0.015 * fs);     % 15 ms overlap
window      = hamming(winLenMfcc);

% Extract MFCC + first and second derivatives -> 39 features/frame
[coeffs, delta, deltaDelta, ~] = mfcc(vadSignal, fs, ...
    'Window', window, 'OverlapLength', overlapMfcc);
features = [coeffs, delta, deltaDelta];
```



4. Model Training (GMM)

Three separate Gaussian Mixture Models — one per class, each with 16 mixture components — model the 39-dimensional feature distribution of "Light On", "Light Off" and "Other". A small regularisation value keeps the covariance estimates numerically stable. The trained models are saved to a .mat file for real-time classification and deployment.

```
numMixtures = 16;
options = statset('MaxIter', 200, 'Display', 'final');

% Train one GMM per class on the pooled 39-D feature frames
gmmOn    = fitgmdist(allFeatures{1}, numMixtures, ...
             'Options', options, 'RegularizationValue', 0.01);
gmmOff   = fitgmdist(allFeatures{2}, numMixtures, ...
             'Options', options, 'RegularizationValue', 0.01);
gmmOther = fitgmdist(allFeatures{3}, numMixtures, ...
             'Options', options, 'RegularizationValue', 0.01);

% Save models for deployment
save(fullfile('models','trained_gmm_models.mat'), ...
     'gmmOn', 'gmmOff', 'gmmOther');
```

5. Testing & Evaluation

During testing, each unseen clip passes through the same pipeline. Its feature frames are scored under all three GMMs using the total log-likelihood, and the clip is assigned to the highest-scoring model. A small constant (1e-10) is added inside the logarithm to avoid taking log(0).

```
% Log-likelihood of the clip under each model
scoreOn    = sum(log(pdf(gmmOn,    features) + 1e-10));
scoreOff   = sum(log(pdf(gmmOff,   features) + 1e-10));
```

```
scoreOther = sum(log(pdf(gmmOther, features) + 1e-10));

% Predicted class = model with the highest score
[~, predIdx] = max([scoreOn, scoreOff, scoreOther]);
```

The system achieves an overall accuracy of 88.27% on the balanced test set. The confusion matrix shows strong recognition of "Light On" and "Other", while "Light Off" is the most challenging class — most often confused with "Light On", which is acoustically similar.

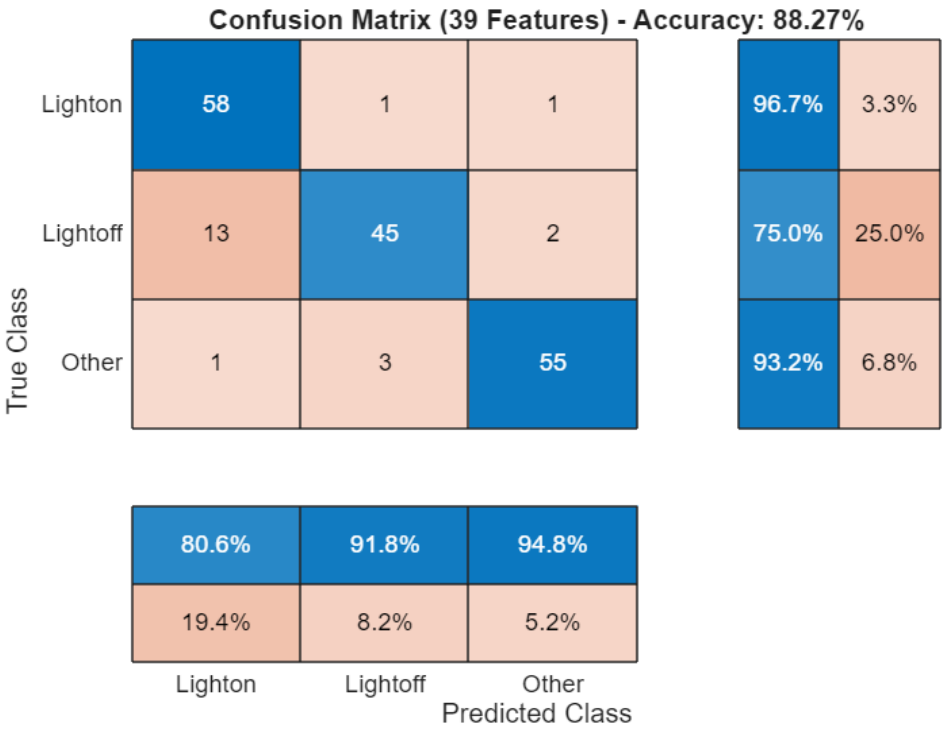


Figure 2. Confusion matrix on the test set (39 features).

Class	Recall (Accuracy)	Note
Light On	96.7%	Strongest class
Light Off	75.0%	Most challenging
Other	93.2%	Strong
Overall	**88.27%**	**Balanced test set (60/class)**

6. Real-Time GUI

A MATLAB App Designer interface deploys the trained models for live use. Pressing START records two seconds of microphone audio, plots its time- and frequency-domain views, and runs the full pre-processing and MFCC pipeline. The three GMM scores are converted into a softmax confidence percentage; the recognised command then drives a virtual light (LED lamp) and is written to a system log.

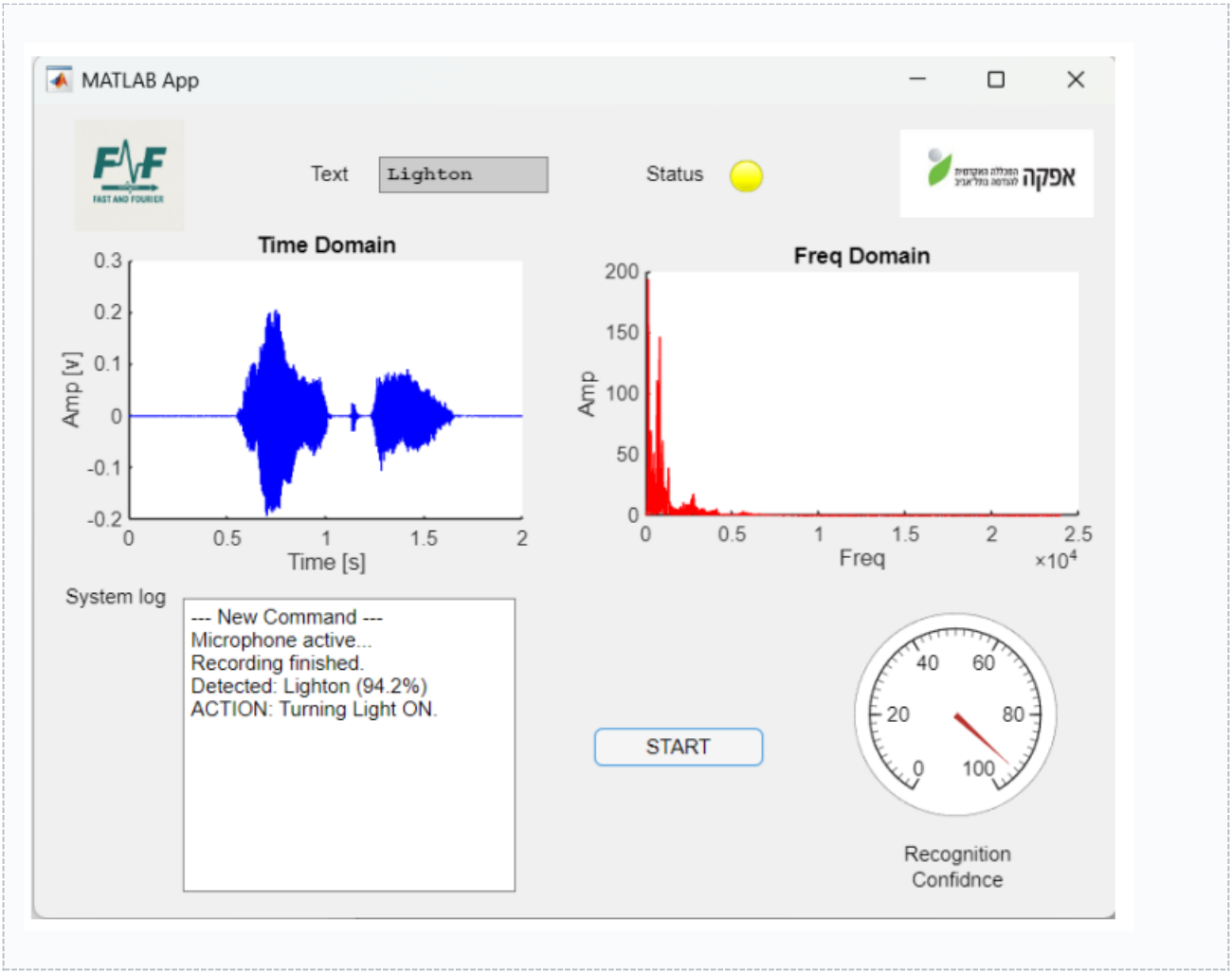


Figure 4. Real-time GUI — recognition output.

7. Conclusion

The project delivers a complete, working voice-command recognition system: a principled DSP front end (pre-emphasis, VAD, windowing), a compact 39-dimensional MFCC feature set, and per-class Gaussian Mixture Models combined with a maximum-likelihood decision rule. At 88.27% overall accuracy with a responsive real-time interface, it demonstrates that classical DSP and statistical modelling remain an effective, transparent approach to small-vocabulary speech recognition. The clearest avenue for improvement is separating the two similar commands, for example with more discriminative features or additional training data for "Light Off".